

Exercise 1

```
Hi again from thread # 0
[ec2-user@ip-172-31-80-143 ~]$ ./exercise1
thread # 1
thread # 2
thread # 3
thread # 0
Outside Barrier # 2
Hi again from thread # 2
Outside Barrier # 1
Hi again from thread # 1
Outside Barrier # 3
Hi again from thread # 3
Outside Barrier # 0
Hi again from thread # 0
[ec2-user@ip-172-31-80-143 ~]$ |
```

The `#pragma omp barrier` directive creates a strict synchronization point in your code where all threads must pause and wait for each other before continuing.

Here is how it controls the execution of the Exercise 1 program:

1. Independent Execution (Before the Barrier): When the program enters the `#pragma omp parallel` region, it spawns multiple threads. Each thread independently fetches its ID and prints ``thread # [id]``. Because threads execute at slightly different speeds, these initial messages print in a random, unpredictable order.
2. The Waiting Phase: As each thread finishes its first `printf` statement, it hits the `#pragma omp barrier`. If a thread finishes early, it is forced to pause at this line. It cannot execute any further code.
3. The Release: The barrier remains closed until the very last, slowest thread finishes its first print and arrives at the barrier. Once all threads are accounted for, the barrier is lifted.

4. Resumed Execution (After the Barrier): All threads are released simultaneously to execute the final two printf statements (Outside Barrier... and Hi again...).

Observable Effect:

Because of this synchronization, your terminal output is strictly divided into two phases. You will see every single `thread # [id]` message grouped at the top. You will never see an Outside Barrier or Hi again message print before all threads have successfully completed the first step.

Exercise 2

```
Hi again from thread # 0
[ec2-user@ip-172-31-80-143 ~]$ export OMP_NUM_THREADS=4
[ec2-user@ip-172-31-80-143 ~]$ ./exercise2
thread # 1
Inside Single # 1
Exiting Single
thread # 2
thread # 3
thread # 0
Hi again from thread # 2
Hi again from thread # 1
Hi again from thread # 3
Hi again from thread # 0
[ec2-user@ip-172-31-80-143 ~]$ |
```

The `#pragma omp single` directive ensures that the enclosed block of code is executed by exactly ONE thread in the parallel team.

1. Parallel Start: All threads print their initial ID.
2. Single Execution: Only ONE chosen thread enters the block to print the "Inside" and "Exiting" messages. All other threads bypass this block.
3. Implicit Barrier: By default, there is an automatic wait point at the end of the single block. The bypassing threads wait there until the chosen thread completes the block.

4. Resumption: Once the block is finished, all threads resume together and print their final "Hi again" messages.

Exercise 3

```
[ec2-user@ip-172-31-80-143 ~]$ export OMP_N
[ec2-user@ip-172-31-80-143 ~]$ ./exercise3
thread # 1
Hi again from thread # 1
thread # 2
Hi again from thread # 2
thread # 3
Hi again from thread # 3
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
[ec2-user@ip-172-31-80-143 ~]$ |
```

The `#pragma omp master` directive ensures that the enclosed block of code is executed ONLY by the master thread (Thread 0).

1. Parallel Start: All threads print their initial ID.
2. Master Execution: Only Thread 0 enters the master block to print the "Inside" and "Exiting" messages. All other threads bypass this block.
3. NO Implicit Barrier: Unlike 'single', there is no automatic wait point at the end of a 'master' block. Other threads bypass the block and immediately continue.
4. Resumption: Because there is no barrier, the bypassing threads will print their final "Hi again" messages without waiting for Thread 0 to finish the master block. Thread 0 prints its final message whenever it finishes the block.

Exercise 4

With collapse(2), the number of outputs were increased.

```
Time taken = 2.763096 seconds
[ec2-user@ip-172-31-80-143 ~]$ nano exercise4.c
[ec2-user@ip-172-31-80-143 ~]$ ./exercise4
r = 5, k = 1, thread # = 3
r = 5, k = 2, thread # = 3
r = 3, k = 1, thread # = 1
r = 3, k = 2, thread # = 1
r = 3, k = 3, thread # = 1
r = 3, k = 4, thread # = 1
r = 3, k = 5, thread # = 1
r = 3, k = 6, thread # = 1
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 4, k = 1, thread # = 2
r = 4, k = 2, thread # = 2
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 4, k = 6, thread # = 2
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 4, k = 9, thread # = 2
r = 4, k = 10, thread # = 2
r = 5, k = 3, thread # = 3
r = 5, k = 4, thread # = 3
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 5, k = 7, thread # = 3
r = 5, k = 8, thread # = 3
r = 5, k = 9, thread # = 3
r = 5, k = 10, thread # = 3
[ec2-user@ip-172-31-80-143 ~]$ client loop: send disconnect
```

Without collapse:

```
[ec2-user@ip-172-31-80-143 ~]$ gcc -fopenmp exercise4.c -o exercise4
[ec2-user@ip-172-31-80-143 ~]$ ./exercise4
r = 3, k = 1, thread # = 1
r = 3, k = 2, thread # = 1
r = 3, k = 3, thread # = 1
r = 3, k = 4, thread # = 1
r = 3, k = 5, thread # = 1
r = 3, k = 6, thread # = 1
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
r = 4, k = 1, thread # = 2
r = 4, k = 2, thread # = 2
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 4, k = 6, thread # = 2
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 4, k = 9, thread # = 2
r = 4, k = 10, thread # = 2
r = 5, k = 1, thread # = 3
r = 5, k = 2, thread # = 3
r = 5, k = 3, thread # = 3
r = 5, k = 4, thread # = 3
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 5, k = 7, thread # = 3
r = 5, k = 8, thread # = 3
r = 5, k = 9, thread # = 3
r = 5, k = 10, thread # = 3
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
[ec2-user@ip-172-31-80-143 ~]$
```

Exercise 5

```
Fibonacci of 20 is 6765
[ec2-user@ip-172-31-80-143 ~]$ export OMP_NUM_THREADS=4
[ec2-user@ip-172-31-80-143 ~]$ ./exercise5
Fibonacci of 20 is 6765
[ec2-user@ip-172-31-80-143 ~]$ |
```

Exercise 6

```
[ec2-user@ip-172-31-80-143 ~]$ gcc -fopenmp exercise6.
[ec2-user@ip-172-31-80-143 ~]$ ./exercise6
Multiplication successful. Test value C[150] = 300
[ec2-user@ip-172-31-80-143 ~]$ |
```

Exercise 7

```
Time taken = 2.875415 seconds
[ec2-user@ip-172-31-80-143 ~]$ export OMP_NUM_THREADS=1
./exercise7
Area of Mandelbrot set = 1.511544
Time taken = 2.764127 seconds
[ec2-user@ip-172-31-80-143 ~]$ export OMP_NUM_THREADS=2
./exercise7
Area of Mandelbrot set = 1.511544
Time taken = 2.758410 seconds
[ec2-user@ip-172-31-80-143 ~]$ export OMP_NUM_THREADS=3
./exercise7
Area of Mandelbrot set = 1.511544
Time taken = 2.762253 seconds
[ec2-user@ip-172-31-80-143 ~]$ export OMP_NUM_THREADS=4
./exercise7
Area of Mandelbrot set = 1.511544
Time taken = 2.763096 seconds
[ec2-user@ip-172-31-80-143 ~]$ |
```